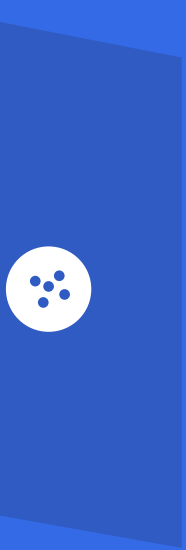
POSTGRESQL ESSENTIALS FOR ODOO DATABASES

PostgreSQL for Odoo

SQL, roles, indexes, and backup basics

Merit Advisory
Odoo Full-Stack Developer Program
Beginner-friendly lecture materials
July 2026



WHAT WE WILL COVER

Agenda

1. Section 1: Install PostgreSQL on Ubuntu to List Databases and Tables — 7 topics
2. Section 2: Data Types Overview to Update Data — 7 topics
3. Section 3: Delete Data to Group Rows with GROUP BY — 7 topics
4. Section 4: Filter Groups with HAVING to Understand the pg_hba.conf Idea — 7 topics
5. Section 5: Understand the postgresql.conf Idea to Keep a Backup Habit for Odoo — 5 topics

1 Section 1

Install PostgreSQL on Ubuntu to List Databases and Tables

1.1 Install PostgreSQL on Ubuntu

- Use apt for a standard Ubuntu install.
- Install contrib tools for common extensions.
- Verify the version after installation.

Install packages

```
sudo apt update
sudo apt install -y postgresql postgresql-contrib
```

Check installed version

```
psql --version
sudo -u postgres psql -c "SELECT version();"
```

1.2 Start and Check the PostgreSQL Service

- PostgreSQL usually runs as a systemd service.
- Enable the service to start after reboot.
- Check status before testing applications.

Start service

```
sudo systemctl enable --now postgresql
```

View status

```
systemctl status postgresql --no-pager
```

1.3 Connect with psql

- psql is the main PostgreSQL command-line client.
- Local admin access often uses the postgres OS user.
- Use \conninfo to confirm the connection.

Connect as postgres

```
sudo -u postgres psql
```

Connect to a database

```
psql -h 127.0.0.1 -U odoo17 -d postgres
\conninfo
```

1.4 Create a PostgreSQL Role

- Roles are database identities.
- LOGIN lets a role connect like a user.
- Give only the privileges the role needs.

Create login role

```
sudo -u postgres createuser --pwprompt app_user
```

Create role with SQL

```
CREATE ROLE app_user LOGIN PASSWORD 'change-me';
```

1.5 Create a Database

- A database stores schemas, tables, and data.
- Set the owner when the database is created.
- Use clear names for training and development.

Create with createdb

```
sudo -u postgres createdb -O app_user training_db
```

Create with SQL

```
CREATE DATABASE training_db OWNER app_user;
```

1.6 Grant Database Permissions

- Grants control what a role can do.
- Database grants are not the same as table grants.
- Review broad privileges before production use.

Grant database access

```
GRANT CONNECT ON DATABASE training_db TO app_user;
```

Grant schema access

```
GRANT USAGE, CREATE ON SCHEMA public TO app_user;
```

1.7 List Databases and Tables

- psql meta commands start with a backslash.
- \l lists databases.
- \dt lists tables in the current database.

List databases

```
\l
```

List tables

```
\c training_db
\dt
```

2 Section 2

Data Types Overview to Update Data

2.1 Data Types Overview

- Choose types that match the meaning of data.
- Use text types for names and descriptions.
- Use numeric and date types for calculations.

Common text and number types

```
name varchar(100)
notes text
quantity integer
price numeric(10, 2)
```

Common date and flag types

```
is_active boolean
created_on date
created_at timestamp
```

2.2 Create a Table

- Tables define columns and constraints.
- Use consistent names for columns.
- Start small and add constraints intentionally.

Simple table

```
CREATE TABLE customers (
    id integer,
    name text,
    email text
);
```

Table with defaults

```
CREATE TABLE notes (
    id integer,
    body text,
    created_at timestamp DEFAULT now()
);
```

2.3 Use a Primary Key

- A primary key uniquely identifies each row.
- It cannot be null.
- Most tables should have one primary key.

Identity primary key

```
CREATE TABLE products (
    id bigint GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    name text NOT NULL
);
```

Natural primary key

```
CREATE TABLE countries (
    code char(2) PRIMARY KEY,
    name text NOT NULL
);
```

2.4 Use a Foreign Key

- A foreign key links one table to another.
- It protects data from broken references.
- Name relationship columns clearly.

Customer orders

```
CREATE TABLE orders (
    id bigint GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    customer_id bigint REFERENCES customers(id)
);
```

Required relationship

```
product_id bigint NOT NULL REFERENCES products(id)
```

2.5 Insert Data

- INSERT adds new rows.
- List columns to avoid order mistakes.
- Use RETURNING to see created values.

Insert one row

```
INSERT INTO customers (name, email)
VALUES ('Ada', 'ada@example.com');
```

Insert and return id

```
INSERT INTO products (name)
VALUES ('Keyboard')
RETURNING id, name;
```

2.6 Read Data with SELECT

- SELECT reads rows from tables.
- Choose only the columns you need.
- Use ORDER BY for predictable order.

Select columns

```
SELECT id, name, email
FROM customers;
```

Sort results

```
SELECT name, price
FROM products
ORDER BY name;
```

2.7 Update Data

- UPDATE changes existing rows.
- Always check the WHERE clause.
- RETURNING can show changed rows.

Update one customer

```
UPDATE customers
SET email = 'ada.new@example.com'
WHERE id = 1;
```

Update and review

```
UPDATE products
SET price = 19.99
WHERE name = 'Keyboard'
RETURNING id, name, price;
```

3 Section 3

Delete Data to Group Rows with GROUP BY

3.1 Delete Data

- DELETE removes rows from a table.
- Use WHERE to target specific rows.
- Test with SELECT before deleting.

Delete one row

```
DELETE FROM customers
WHERE id = 10;
```

Preview then delete

```
SELECT * FROM products WHERE name = 'Old Item';
DELETE FROM products WHERE name = 'Old Item';
```

3.2 Filter Rows with WHERE

- WHERE limits which rows are returned.
- Combine filters with AND and OR.
- Use parentheses for clear logic.

One condition

```
SELECT * FROM customers
WHERE email IS NOT NULL;
```

Multiple conditions

```
SELECT * FROM products
WHERE price > 10 AND is_active = true;
```

3.3 Search Text with LIKE and ILIKE

- LIKE matches text patterns.
- ILIKE ignores letter case.
- The percent sign matches any text.

Case-sensitive search

```
SELECT name FROM customers
WHERE name LIKE 'A%';
```

Case-insensitive search

```
SELECT name FROM customers
WHERE name ILIKE '%smith%';
```

3.4 Filter with IN and BETWEEN

- IN checks a value against a list.
- BETWEEN checks an inclusive range.
- These filters keep SQL readable.

Use IN

```
SELECT * FROM orders
WHERE status IN ('draft', 'sent');
```

Use BETWEEN

```
SELECT * FROM invoices
WHERE amount_total BETWEEN 100 AND 500;
```

3.5 Combine Rows with INNER JOIN

- INNER JOIN returns matching rows only.
- Join conditions usually compare keys.
- Use table aliases for shorter SQL.

Orders with customers

```
SELECT o.id, c.name
FROM orders o
INNER JOIN customers c ON c.id = o.customer_id;
```

Lines with products

```
SELECT l.id, p.name
FROM order_lines l
JOIN products p ON p.id = l.product_id;
```

3.6 Keep Rows with LEFT JOIN

- LEFT JOIN keeps all rows from the left table.
- Missing matches appear as null values.
- It is useful for optional relationships.

Customers with optional orders

```
SELECT c.name, o.id AS order_id
FROM customers c
LEFT JOIN orders o ON o.customer_id = c.id;
```

Products with optional categories

```
SELECT p.name, c.name AS category
FROM products p
LEFT JOIN categories c ON c.id = p.category_id;
```

3.7 Group Rows with GROUP BY

- GROUP BY collects rows into groups.
- Use aggregates to summarize each group.
- Every selected non-aggregate column must be grouped.

Orders by status

```
SELECT status, COUNT(*)
FROM orders
GROUP BY status;
```

Sales by customer

```
SELECT customer_id, SUM(amount_total)
FROM invoices
GROUP BY customer_id;
```

4 Section 4

Filter Groups with HAVING to Understand the pg_hba.conf Idea

4.1 Filter Groups with HAVING

- HAVING filters grouped results.
- WHERE filters rows before grouping.
- Use HAVING with aggregate conditions.

Customers with many orders

```
SELECT customer_id, COUNT(*)
FROM orders
GROUP BY customer_id
HAVING COUNT(*) > 3;
```

Large sales groups

```
SELECT customer_id, SUM(amount_total)
FROM invoices
GROUP BY customer_id
HAVING SUM(amount_total) >= 1000;
```

4.2 Use COUNT, SUM, and AVG

- COUNT counts rows or values.
- SUM adds numeric values.
- AVG calculates an average.

Count rows

```
SELECT COUNT(*) AS order_count
FROM orders;
```

Sales summary

```
SELECT SUM(amount_total), AVG(amount_total)
FROM invoices;
```

4.3 Create an Index

- Indexes help PostgreSQL find rows faster.
- Index columns used often in filters or joins.
- Too many indexes slow writes.

Index a search column

```
CREATE INDEX idx_customers_email ON customers (email);
```

Index a foreign key

```
CREATE INDEX idx_orders_customer_id ON orders (customer_id
```

4.4 Read Plans with EXPLAIN ANALYZE

- EXPLAIN shows how PostgreSQL plans a query.
- ANALYZE runs the query and measures time.
- Use it carefully on changing queries.

Inspect a select

```
EXPLAIN ANALYZE
SELECT * FROM customers WHERE email = 'ada@example.com';
```

Inspect a join

```
EXPLAIN ANALYZE
SELECT o.id, c.name
FROM orders o
JOIN customers c ON c.id = o.customer_id;
```

4.5 Back Up with pg_dump

- pg_dump creates a logical backup.
- Custom format is flexible for restore.
- Store backups outside the database server.

Custom format backup

```
pg_dump -h 127.0.0.1 -U odoo17 -Fc -f odoo17.dump odoo17_p
```

Plain SQL backup

```
pg_dump -U odoo17 -f training_db.sql training_db
```

4.6 Restore with pg_restore

- pg_restore reads custom-format dumps.
- Restore into the correct target database.
- Test restores before an emergency.

Create target database

```
createdb -h 127.0.0.1 -U odoo17 odoo17_restore
```

Restore dump

```
pg_restore -h 127.0.0.1 -U odoo17 -d odoo17_restore odoo17
```

4.7 Understand the pg_hba.conf Idea

- pg_hba.conf controls client authentication.
- Rules are checked from top to bottom.
- Reload PostgreSQL after safe changes.

Local password rule

```
host    all    odoo17    127.0.0.1/32    scram-sha-256
```

Reload configuration

```
sudo systemctl reload postgresql
```

5 Section 5

Understand the postgresql.conf Idea to Keep a Backup Habit for Odoo

5.1 Understand the postgresql.conf Idea

- postgresql.conf controls server behavior.
- Common settings include port and listen address.
- Some changes require restart, not reload.

Listen locally

```
listen_addresses = 'localhost'
port = 5432
```

Show config file

```
SHOW config_file;
SHOW listen_addresses;
```

5.2 See Active Connections

- pg_stat_activity shows current sessions.
- It helps identify users, databases, and queries.
- Filter it when many sessions exist.

View active sessions

```
SELECT pid, usename, datname, state
FROM pg_stat_activity;
```

Find Odoo sessions

```
SELECT pid, state, query
FROM pg_stat_activity
WHERE usename = 'odoo17';
```

5.3 Cancel or Terminate a Query

- Cancel asks a query to stop cleanly.
- Terminate closes the whole session.
- Confirm the PID before acting.

Cancel a query

```
SELECT pg_cancel_backend(12345);
```

Terminate a session

```
SELECT pg_terminate_backend(12345);
```

5.4 Use a Dedicated Odoo Database User

- Do not run Odoo as the postgres superuser.
- Use one clear role for each Odoo environment.
- Grant CREATEDB only when it is needed.

Development role

```
CREATE ROLE odoo17_dev LOGIN CREATEDB PASSWORD 'change-me'
```

Connection options

```
db_user = odoo17_dev
db_password = change-me
db_host = 127.0.0.1
```

5.5 Keep a Backup Habit for Odoo

- Back up before upgrades and migrations.
- Keep database and filestore backups together.
- Practice restoring backups regularly.

Database backup

```
pg_dump -U odoo17 -Fc -f before_upgrade.dump odoo17_prod
```

Filestore backup

```
tar -czf filestore_before_upgrade.tgz ~/.local/share/Odoo/
```

NEXT STEP

Try the examples on your machine

Then open the next module deck

Merit Advisory

03 · Database